



**Baptiste Cojean, Gatien Auffret, Aubin
Bergeret, Matéo Serandour et Jules
Renouard**

ESIR 2 INFO

**Documentation
utilisateur du compilateur
et du langage**

19/01/2026

Sommaire

Sommaire.....	2
Comment écrire un programme en langage WHILE.....	2
Syntaxe des expressions.....	2
Commandes de base.....	2
Structure d'un programme.....	2
Exemples de réalisation.....	3
Comment utiliser le compilateur.....	3

Comment écrire un programme en langage WHILE

Le langage WHILE permet la manipulation de données structurées sous forme d'arbres binaires. La rédaction d'un programme repose sur une syntaxe stricte composée de commandes et d'expressions.

Syntaxe des expressions

Les expressions représentent les valeurs manipulées par le programme. Elles se déclinent selon trois formes :

- NIL : Représente l'arbre vide (ou la valeur constante zéro).
- Variable : Identifiée par une chaîne de caractères débutant par une majuscule (exemple : X, Resultat).
- (cons E1 E2) : Construit un nouvel arbre à partir de deux expressions E1 et E2.

Commandes de base

Le flux d'exécution s'articule autour des structures suivantes :

- Affectation : Variable := Expression. Cette commande attribue la valeur d'une expression à une variable.
- Séquence : Les commandes successives sont séparées par un point-virgule (;).
- Conditionnelle : if Expression then Commande1 else Commande2 fi. Le programme exécute Commande1 si l'expression est différente de NIL, sinon il exécute Commande2.
- Boucle : while Expression do Commande od. La commande se répète tant que l'expression ne vaut pas NIL.

Structure d'un programme

Un programme WHILE se définit par un nom, des entrées et des sorties. La structure respecte le schéma suivant :

```
function nom_du_programme :  
read Entree1, Entree2  
%  
  Corps du programme (Commandes)  
%  
write Sortie1
```

Exemples de réalisation

1. Calcul de la fonction identité : Ce programme reçoit une entrée et la restitue sans modification.

```
function identite :  
read X  
%  
    Resultat := X  
%  
write Resultat
```

2. Succession de valeurs (Incrémentation) : Ce programme utilise la structure de l'arbre pour représenter une progression.

```
function suivant :  
read X  
%  
    X := (cons NIL X)  
%  
write X
```

3. Boucle de parcours : Ce programme vide une liste structurée.

```
function vider_liste :  
read Liste  
%  
    while Liste do  
        Liste := (tl Liste)  
    od  
%  
write Liste
```

Comment utiliser le compilateur

Notre compilateur traduit le code source WHILE vers du code C++, le compile, et produit un exécutable natif optimisé.

Avant tout, il faut donner les droit d'exécution au fichier whilec.sh, pour cela, exécutez la commande suivante dans votre terminal bash : `chmod +x whilec.sh`

Il faut ensuite lancer simplement le script whilec.sh. Ce script compile d'abord les fichiers java permettant d'avoir l'Arbre de Syntaxe Abstraite, l'analyse sémantique, le code 3 adresse avant et après optimisation.

Il permet aussi d'obtenir l'équivalent en C++ du code while fourni lors de l'exécution.

Il permet également de démarrer le compilateur qui compile le langage C++ fourni par le backend et vérifie qu'il est correct.

Récapitulatif des commandes à lancer depuis la racine du projet :

```
chmod +x whilec.sh  
./whilec.sh <fichier_while_optionnel.txt>
```